API REST de gestión de tareas

Freddy Potes | Universidad Politécnica Salesiana
Aplicaciones Web · Cuarto nivel · Unidad 2 · 1 de noviembre de 2026

Objetivo y solución implementada

Se desarrolló una API para crear, consultar, actualizar y eliminar tareas con Node.js 24, Express 5.1.0 y SQLite. Incluye exportación CSV y notificaciones en tiempo real mediante Server-Sent Events (SSE). Las 16 pruebas automatizadas aprobaron; verifican el contrato HTTP, la persistencia y los casos de error. La función de archivo satisface la opción «archivo o sesión» de la consigna.

Arquitectura y diseño funcional

El flujo separa HTTP (app.js), casos de uso (service.js), persistencia (repository.js) y eventos (events.js). domain.js agrupa funciones puras de validación, normalización y conversión CSV; devuelve nuevos valores sin modificar la entrada. server.js configura el archivo de base de datos y el cierre del proceso. Las dependencias se inyectan para aislar las pruebas.

Recursos y contrato HTTP

Método	Ruta	Respuesta principal
GET	/api/tasks	200 · Lista de tareas
POST	/api/tasks	201 · Location y ETag
GET	/api/tasks/:id	200 · Tarea y ETag
PATCH	/api/tasks/:id	200 · Nueva versión; exige If-Match
DELETE	/api/tasks/:id	204 · Sin cuerpo; exige If-Match
GET	/api/tasks/export.csv	200 · Archivo tareas.csv
GET	/api/events	200 · Flujo text/event-stream

Persistencia validación y conflictos

tasks almacena id, title, title_key, description, status, version y fechas UTC. El título admite 1 a 120 caracteres; la descripción, hasta 1000; los estados son pending, in_progress y done. Se rechazan campos desconocidos, controles y cuerpos mayores de 16 KiB. SQLite aplica CHECK y UNIQUE; las consultas usan parámetros, sin concatenar entradas (Node.js, s. f.).

Las escrituras usan transacciones y control de versión. PATCH y DELETE requieren If-Match con el ETag consultado: versión antigua → 412; cabecera ausente → 428; título duplicado → 409. Los errores comparten {error: {code, message, details}}; también se controlan 400, 404, 405, 413, 415 y 422. La capa HTTP usa los mecanismos de Express (s. f.).

Archivo y comunicación en tiempo real

La exportación CSV usa nombre fijo, UTF-8, celdas entre comillas y escape de comillas internas. Ante valores que empiezan con =, +, - o @, añade un apóstrofo para impedir su interpretación como fórmulas. No admite rutas de archivo proporcionadas por el cliente.

SSE emite task.changed con action, task y emittedAt después del COMMIT para altas, ediciones y borrados. Envía heartbeat cada 15 segundos, limita a 50 conexiones y libera clientes desconectados. Es un canal unidireccional (MDN Web Docs, s. f.); al reconectar debe consultarse la colección, pues los eventos no mantienen historial.

Ejemplo de petición respuesta y conflicto

```
POST /api/tasks   Content-Type: application/json
{"title":"Preparar entrega"}
```

Devuelve 201, Location: /api/tasks/1 y ETag: "1". El cuerpo data contiene la tarea con status: "pending", version: 1 y fechas UTC. PATCH /api/tasks/1 con If-Match: "1" y {"status":"done"} devuelve 200 y ETag: "2". Reutilizar "1" provoca:

```
HTTP 412
{"error":{"code":"VERSION_CONFLICT",
"message":"La tarea cambió. Consulte la versión actual antes de reintentar.",
"details":[]}}
```

Pruebas y resultados

Se ejecutaron 16 pruebas con node:test y peticiones HTTP reales: CRUD; validaciones y límites; duplicados y rollback; identificación y precondiciones; dos ediciones simultáneas; inyección SQL como dato; persistencia al reabrir SQLite; CSV; eventos, desconexión y límite SSE. Resultado: 16 aprobadas y 0 fallidas. Además, una demostración automática verificó 10 peticiones y registró sus respuestas.

Evidencias en el repositorio: test/api.test.js, docs/resultados-pruebas.txt y docs/demo-resultados.json. docs/solicitudes.http y README.md incluyen peticiones, respuestas y pasos de ejecución. No se realizaron pruebas de carga ni se calcula un porcentaje de cobertura.

Reproducción y alcance

Con Node.js 24.x, ejecutar npm ci, npm test y npm start. SQLite se crea en data/tasks.sqlite. Para repetir las diez peticiones: node scripts/demo.js. La demostración es local en http://127.0.0.1:3000/health; los eventos se observan con curl.exe -N http://127.0.0.1:3000/api/events.

La aplicación escucha solo en el equipo local, sin autenticación ni despliegue público. SQLite se adoptó al no indicarse otro motor en la consigna. Para uso multiusuario se requieren autenticación, autorización, HTTPS y paginación. El resultado integra las funciones solicitadas con persistencia y manejo comprobado de conflictos.

Repositorio github.com/efrain-svg/Potes_Freddy_AplWeb_U2

Referencias

Express. (s. f.). 5.x API reference. https://expressjs.com/en/api/

Node.js. (s. f.). SQLite. https://nodejs.org/docs/latest-v24.x/api/sqlite.html

MDN Web Docs. (s. f.). Using server-sent events. developer.mozilla.org